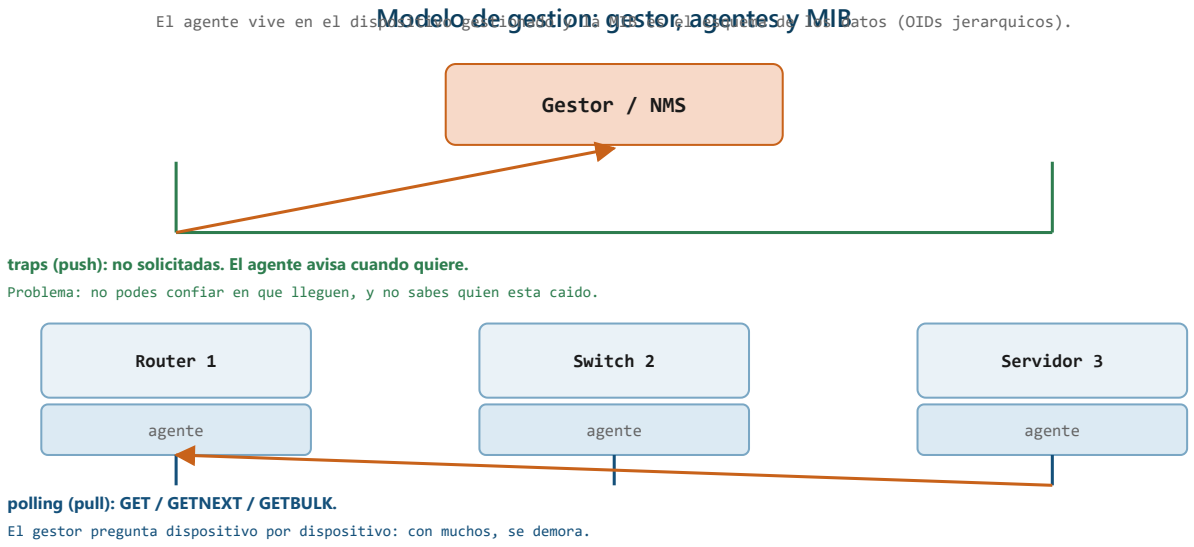
Clase 46 — Gestión de redes: SNMP, NETCONF y YANG

Bloque: F (Servicios y gestión) · **Semana:** 16 · **Duración estimada:** 120 min **Correlativa:** Clase 45 (QoS, shaping, MPLS) · **Sigue con:** Clase 47 (Seguridad) **Material de referencia:** Tanenbaum cap. 5 §5.5 (gestión de redes) · `ACTUALIZACIONES.md` §11

🎯 Al terminar esta clase vas a poder

1. Explicar por qué una red grande **no se puede administrar con `ping`**.
2. Distinguir **agente** de **gestor** y describir qué hace cada uno, con la asimetría entre "coleccionar" y "decidir".
3. Describir el modelo **MIB**: qué es un OID, por qué es un **árbol**, y por qué los OIDs no son nombres legibles.
4. Explicar **polling** y **traps**, con la comparación de coste que se pregunta en todos los exámenes.
5. Enumerar las **5 operaciones del SNMP** y decir para qué sirve cada una.
6. Explicar por qué **SNMPv1/v2c** con *community strings* es inaceptable y qué arregla **SNMPv3** (USM + VACM).
7. Describir **NETCONF** y **YANG** y decir **en qué se diferencian de SNMP** — la comparación que es el centro de la clase.



> **Prerequisito del examen:** `2010-03 P4` (8 pts — agente, gestor y modelo MIB), `2008-07 P4` (4 pts — plano de datos vs. plano de control; en banda vs. fuera de banda), `2011-02 P1` (la gestión aparece como parte del escenario de TLS).

PARTE 1 — TEORÍA

1.1 El problema: por qué `ping` no es gestión

Imaginate la red de una universidad: 300 switches, 80 routers, 2 000 PCs, 40 APs. Son **~2 400 dispositivos**. ¿Cómo sabés si alguno está caído?

La respuesta ingenua —"hago `ping` a todos"— falla por tres razones que conviene saber enunciar:

Problema	Detalle
Escala	2 400 pings por minuto es 1 línea de terminal por dispositivo. No escala a nada.
No dice nada útil	`ping` dice "responde" o "no responde". No dice *cuánta* CPU usa, *cuánta* memoria, *qué interfaces están arriba*, *cuánto tráfico pasó por el puerto 7*.
No hay histórico	`ping` es instantáneo. Si el puerto 7 se congestionó **a las 3:14 y volvió a la normalidad a las 3:16**, nadie se entera salvo que alguien estuviera mirando.

La gestión de redes es la disciplina de responder tres preguntas sobre una red con cientos de dispositivos:

```
ESTADO      ¿qué está pasando AHORA?
HISTORIA    ¿qué pasó AYER a las 3:14?
CONTROL     ¿qué le MANDO que haga?
```

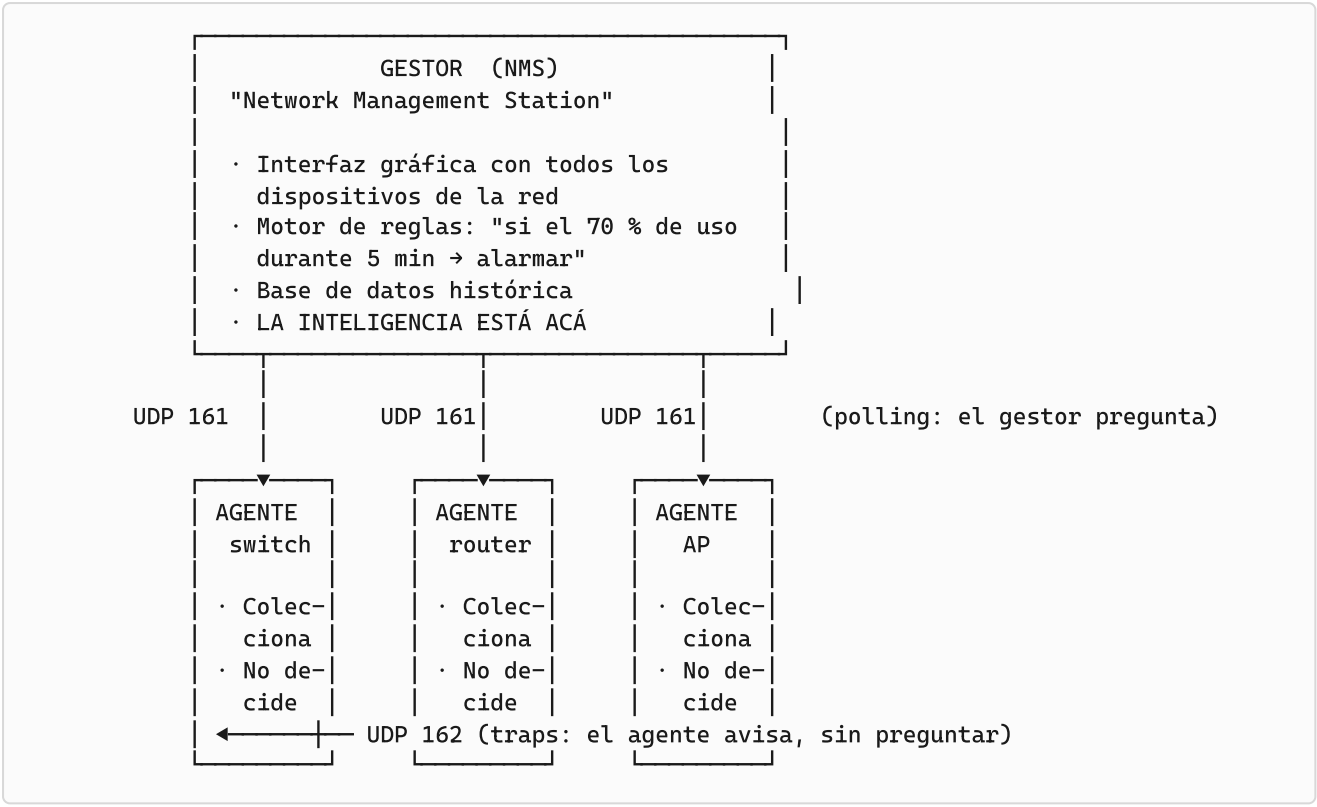
Y para eso hacen falta **cinco funciones** (RFC 1597, el modelo clásico):

```
┌──────────────────────────────────────────────────────────────┐
│  1. FAILURE           detectar que algo falló                │
│  2. CONFIGURATION     cambiar la configuración a distancia   │
│  3. ACCOUNTING         medir quién usó qué                   │
│  4. PERFORMANCE        medir eficiencia y recursos           │
│  5. SECURITY           controlar quién puede hacer todo lo   │
│                         anterior                             │
└──────────────────────────────────────────────────────────────┘
```

> **La definición que hay que saber de memoria:** *la gestión de redes es el conjunto de protocolos y procedimientos que permiten observar, controlar y modificar una red de forma centralizada y estandarizada, con independencia del fabricante del equipo.*

1.2 El modelo SNMP: dos entidades y un idioma común

SNMP (*Simple Network Management Protocol*) es la respuesta. Y es una respuesta **deliberadamente simple**: un solo protocolo de aplicación que corre sobre **UDP**, normalmente en los puertos **161** (agente) y **162** (traps).



La asimetría fundamental (esto es lo que se pregunta)

El **agente** es un programa **pequeño** que vive **dentro** del equipo. Y esa es toda la historia:

El agente tiene recursos mínimos. Un switch de acceso de 24 puertos tiene procesador lento, memoria contada y flash de 8 MB. No puede correr una base de datos ni un motor de reglas. Solo puede:

1. **Recoger** valores de sus variables internas (contadores de tráfico, estado de puertos, temperatura, uso de CPU).
2. **Responder** lo que le pregunten por puerto.
3. **Generar notificaciones** (`traps`) cuando un umbral se supera.

El agente NO decide. No dice "la memoria está al 80 %, voy a bajar las peticiones de log". **Decide el gestor.** Y hay una excepción clásica que la cátedra menciona y vale la pena conocer: **RMON** (*Remote Network Monitoring*), donde el agente sí tiene lógica: guarda estadísticas históricas **en el propio dispositivo** para que no haya que traerlas. Es la excepción, y por eso la mencionan los exámenes.

El gestor es un monstruo de recursos. Corre en un servidor con CPU, disco y memoria de sobra. Ahí van **todos** los datos, **todos** los históricos y **toda** la lógica.

	Agente	Gestor (NMS)
Dónde corre	**dentro** del dispositivo	en un servidor central
Recursos	mínimos (CPU, RAM y disco chicos)	abundancia
Qué hace	**coleciona** y **responde**	**interpreta**, **decide**, **actúa**
Cuántos	**uno por dispositivo** (~2 400)	**uno para toda la red**
Almacena histórico	no (salvo RMON)	**sí**, es su razón de ser
Intelligencia	nula	**toda**
Fábrica	del fabricante del equipo (o del SO)	de terceros (Nagios, Zabbix, LibreNMS, SolarWinds, CiscoWorks)

> **La frase para el examen:** *"el agente está limitado por los recursos del dispositivo que administra; por eso su papel es mecánicamente el de collectar, y toda la inteligencia está en el gestor, que es quien centraliza, correlaciona y decide sobre miles de agentes a la vez."*

El problema de la interoperabilidad

Los agentes los pone **cada fabricante**, y cada uno con su versión de firmware. Si el gestor no entiende el idioma de cada uno, no hay gestión. De ahí la necesidad de un **modelo de información estándar**: la **MIB**.

1.3 La MIB: un árbol con nombres numéricos

MIB = *Management Information Base*. Es **la base de datos** de lo que un dispositivo sabe de sí mismo.

```
AGENTE
  │
  │  "no sé lo que es 'ifInOctets', pero sí sé que
  │   tengo la variable número 1.3.6.1.2.1.2.2.1.10"
  ▼
MIB  ─── un archivo de texto, tipo ASN.1, que DEFINE
           la estructura del árbol
```

El punto clave y contraintuitivo: SNMP no manda el nombre, manda el número.

Cada variable de la MIB tiene un **OID** (*Object Identifier*): una **dirección jerárquica** en un árbol universal. El OID es una secuencia de enteros, y cada nivel es un *nodo* del árbol.

```
                iso (1)                          ← la raíz
                  |
               org (1.3)                         ← todos los-fire
                  |
              dod (1.3.6)                        ← Departamento de Defensa (USA)
                  |
              internet (1.3.6.1)                  ← ¡el mundo real empieza acá!
                  |
               mgmt (1.3.6.1.2)                  ← administración
                  |
            mib-2 (1.3.6.1.2.1)                  ← ★ MIB-II: todo IPv4
                  |
                  ├── 1  system ............... info del dispositivo
                  |          ├── 1  sysDescr        "Cisco WS-C2960..."
                  |          ├── 2  sysObjectID     el modelo
                  |          ├── 3  sysUpTime       segundos desde el boot
                  |          ├── 4  sysContact      quién lo administra
                  |          └── 5  sysName         "r1.miuy.edu.uy"
                  |
                  ├── 2  interfaces ........... todas las interfaces
                  |          └── 2.2.1 ifTable
                  |                ├── 10  ifInOctets   ← bytes que entraron
                  |                └── 16  ifOutOctets  ← bytes que salieron
                  |
                  ├── 4  ip ................... todo IPv4
                  |          ├── 9  ipRouteTable     ← la tabla de ruteo
                  |          └── 26 ipNetToMediaTable ← la tabla ARP
                  |
                  └── 8  snmp ................. sobre SNMP mismo
```

El OID `1.3.6.1.2.1.2.2.1.10` significa: en el nodo raíz `iso`, bajá al hijo `org`, después a `dod`, después a `internet`, después a `mgmt`, después a `mib-2`, después a `interfaces`, después a `ifTable`, después al índice `10`, y tomá la variable `ifInOctets`.

El `1.3.6.1` **de** `internet` **es el "cuello de botella"** y hay que saberlo: es literalmente el prefijo de **todas** las MIB de Internet. Si tenés un OID que empieza por `1.3.6.1`, es estándar. Los DHCP (`1.3.6.1.2.1.58.4`) y los bridges (`1.3.6.1.2.1.17`) están colgando de ahí.

Y las empresas cuelgan de `1.3`**:**

```
1.3.6.1.4.1.<N>.<sub>       ← "empresa" (private enterprise)
    |   |   |
    |   |   └── sub-objeto definido por el fabricante
    |   └────── número de la empresa ( Cisco = 9,
    |           Juniper = 2636, HP = 11 )
    └────────── "private enterprise": todo lo no-estándar
```

> **El detalle fino:** las MIB de los fabricantes cuelgan de `private enterprise` porque **IANA reserva ese espacio**. Es lo que permite que un fabricante extienda el modelo sin romper el estándar. Y por eso las MIB de fabricante **no son portables**: el gestor necesita la MIB del fabricante instalada para poder traducir los OIDs a nombres.

El `ifIndex`**, el detalle que se pregunta:** el OID `...ifTable.10` termina en `.10` pero ese `10` es **la variable** (`ifInOctets`), no un índice. La tabla tiene **una fila por interfaz**, y cada fila se direcciona **agrego**

un número al final: `...ifInOctets.1`, `...ifInOctets.2`, `...ifInOctets.3`. Ese número final es el **ifIndex**, y **cambia** si el SO renumera las interfaces. Es una fuente clásica de dolor de cabeza en el monitoreo.

1.4 Pull y push: polling vs. traps

Acá está **el concepto central** de toda la clase, y es la pregunta clásica:

```
POLLING  (pull, "tirón")          TRAPS  (push, "empujón")
─────────────────────────         ──────────────────────────
El GESTOR pregunta                El AGENTE avisa

GET / GETNEXT  ──→ agente         ←── TRAP   agente

· El gestor decide QUÉ y CUÁNDO   · El agente decide CUÁNDO avisar
· Simple, pero genera tráfico     · Reactivo: llega justo cuando pasa
  constante incluso sin cambios     algo
· Si el gestor está caído,          · Se pierde si el gestor está caído
  NO HAY NADA                       (no hay reintento)
· Retardo: hasta un período       · Retardo: casi cero
  completo
```

El coste del polling, que es el argumento que hay que dar

Si tenés **2 400 dispositivos** y hacés polling cada **60 segundos**,eso son:

```
2 400 dispositivos / 60 s = 40 consultas POR SEGUNDO
40 × 3600 = 144 000 consultas POR HORA
× 24 horas = 3 456 000 consultas POR DÍA
```

Tres millones y medio de paquetes al día, para preguntar "¿seguís vivo?" cuando nada cambió. Y encima: **si el gestor se cae, nadie se entera**, porque nadie pregunta. El fallo del monitoreo es silencioso.

El coste del trap

El trap es **una sola cosa**: como máximo **2 traps por segundo por agente**, y **solo si hay algo que reportar**. Es infinitamente más barato.

El problema del trap: es un **UDP**, o sea un datagrama **sin confirmar**. Si el gestor estaba caído cuando ocurrió el evento, **el trap se perdió para siempre**. No hay reintento, no hay cola.

> **La respuesta completa al examen** *—los traps no reemplazan al polling, lo complementan.* **El polling** es el que te dice cómo está la red **en general** y guarda el **histórico**; **el trap** es el que te dice **qué cambió ahora**, y como es *push*, es instantáneo y barato. La combinación clásica es: **polling lento** (cada 5 minutos, para tener la línea de base y detectar dispositivos muertos) **+ traps** (para reaccionar a los eventos). Si hacés polling cada 10 segundos para "no perderte nada", estás pagando caro; y si hacés solo traps, **no detectás un equipo que se apagó** (no hay trap de "me apagé", porque el agente ya está muerto).*

1.5 Las 5 operaciones

SNMPv1/v2c, que es lo que se pregunta:

#	Operación	Tipo	Qué hace
1	**GET**	**pull**	leer **un** OID exacto
2	**GETNEXT**	**pull**	leer **el siguiente** OID del árbol a partir de uno
3	**GETBULK** (v2c)	**pull**	leer **muchos** OIDs de una vez (equivale a N GETNEXT, pero en **un solo** datagrama)
4	**SET**	**push desde el gestor**	**escribir** un valor → es la única forma de **configurar** por SNMP
5	**TRAP**	**push desde el agente**	el agente avisa de un evento

El detalle de `GETNEXT` que hay que entender bien, porque es la clave de cómo funciona el monitoreo:

```
GETNEXT(...ifInOctets)  →   ¿cuál es el OID siguiente en el árbol?

El gestor NO sabe de antemano qué interfaces existen.
Entonces va "recorriendo" el árbol:

  GETNEXT(.1.3.6.1.2.1.2.2.1.10)        =  .1.3.6.1.2.1.2.2.1.10.1  ← if #1
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.1)      =  .1.3.6.1.2.1.2.2.1.10.2  ← if #2
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.2)      =  .1.3.6.1.2.1.2.2.1.10.3  ← if #3
  GETNEXT(.1.3.6.1.2.1.2.2.1.10.3)      =  .1.3.6.1.2.1.16.1  ← ifOutOctets #1
                                              ↑ se pasó a la SIGUIENTE variable
  ...
  GETNEXT(último OID conocido)          =  ERROR  (finSubId)
```

O sea: `GETNEXT` es un puntero que recorre el árbol en orden alfabético-numérico. Y `GETBULK` es el mismo recorrido, pero **N de ellos en un solo datagrama** — que existe justamente porque con 24 puertos y 30 variables son **720 OIDs** por dispositivo, y hacerlos de a uno sería insostenible.

Y la `community string`: en SNMPv1 y v2c, el "autenticado" es una cadena de texto plana — `public` o `private` — que viaja **en claro** en cada paquete. No es cifrado, no es autenticación real, es una contraseña de texto. Es exactamente el punto de entrada de la clase 47.

1.6 SNMPv3: USM y VACM

SNMPv3 no agrega una operación: **rediseñó la seguridad y la autorización**. Y tiene exactamente **dos mecanismos**:

USM — *User-based Security Model*

Responde a **"¿quién sos y podés mandar esto?"**:

Pieza	Qué hace
Autenticación	HMAC sobre el mensaje: si alguien modifica un paquete en el camino, el HMAC no da más y el receptor lo rechaza
Cifrado	el contenido va cifrado (DES clásico, hoy AES-192/256)
No repudio	digest de clave: el emisor manda un hash de su clave para que el receptor verifique que la clave es la misma

Los algoritmos los define una OID que **el propio mensaje declara**:

```
msgAuthoritativeEngineID    → quién es la autoridad
msgUserName                 → el usuario
msgAuthenticationParameters → el HMAC (autenticación)
msgPrivacyParameters        → el vector de cifrado
msgSecurityParameters       → todo lo anterior, con longitudes
```

El detalle fino que vale la pena: **el receptor tiene que responder con su propio parámetro de seguridad, no con el del emisor**. Por eso el OID de algorithms no va en la respuesta: si fuera, un atacante podría hacer que el receptor use un algoritmo débil. Es un detalle de diseño elegante que muestra que el protocolo está pensado contra un atacante activo.

VACM — *View-based Access Control Model*

Responde a **"¿qué podés ver y hacer?"**. Es un **control de acceso** por **vistas** sobre la MIB:

```
┌──────────────────────────────────────────────────┐
│  Vista "solo-interfaces"                         │
│   si     ifview    = 1.3.6.1.2.1.2.2.*  (ifTable)    │
│    permitir GET y GETNEXT                        │
│    NO permitir SET                               │
├──────────────────────────────────────────────────┤
│  Vista "admin-completo"                          │
│    permitir TODO sobre 1.3.6.1.2.1.*             │
│    INCLUDING la community "escritura"            │
└──────────────────────────────────────────────────┘

El usuario "readonly"   →  vista "solo-interfaces"
El usuario "admin-red"  →  vista "admin-completo"
El usuario "sensor"     →  sin acceso a ipRouteTable (que
                           es información sensible)
```

La idea clave: es **principio de mínimo privilegio aplicado a la MIB**. Un sensor térmico no necesita ver la tabla de ruteo, y no por cortesía: la tabla de ruteo es **información muy útil para un atacante**. Separar vistas **reduce la superficie de ataque**.

1.7 ⚠️ Actualización a 2026: NETCONF y YANG

Acá viene la parte que cambia el cuadro, y es importante ser honesto con ella.

Qué es NETCONF

NETCONF (`RFC 6241`, 2011) es un protocolo de **configuración remota** sobre:

- **SSH** como transporte (puerto **830**) → **autenticado y cifrado de fábrica**, sin *community strings* en claro.
- **XML** como codificación.
- **RPCs explícitos**, no GET/SET genéricos.

```
SNMP v2c                          NETCONF
────────                          ────────
GET   / SET   (genéricos)         <get>  <get-config>  <edit-config>
                                 <copy-config>  <delete-config>
                                 <lock>  <unlock>  <commit>  <commit-discard>
No transaccional                  <edit-config> + <commit> = TRANSACCIONAL
No hay "candidato"                datastores: running / candidate / startup
Semántica defined por el agente   semántica DEFINIDA POR EL MODELO (YANG)
```

Los 3 datastores son la diferencia operacional más importante:

```
startup  ── la config con que arrancó el equipo (de fábrica o de un respaldo)
   │
   ▼
candidate ── un BORRADOR. Podés editarlo, validarlo y rechazarlo
   │          sin tocar nada que esté corriendo.
   │          ★ Esto es lo que permite "aplicar" una config de a un
   │            solo paso, y si falla, no queda nada a medias.
   ▼
running ── la config REALMENTE en ejecución ahora

<commit>            candidate  →  running   (con validación previa)
<commit-discard>    <aborta todos los cambios no commiteados>
<lock> / <unlock>   evita que dos administradores editen a la vez
```

> **El `lock` es la clave de por qué existe la semántica transaccional**: en SNMP, un `SET` es un `SET`; si mandás 20 SET y el 15º falla, **los 14 anteriores ya están aplicados** y el dispositivo quedó en un estado intermedio que nadie pidió. Con NETCONF, los 20 van al `candidate`, se validan **todos juntos**, y recién después el `commit` los aplica. **O se aplican todos o ninguno.**

Qué es YANG

YANG (`RFC 7950`, 2016, *Internet Standard*) es el **lenguaje de modelado** de la configuración. Se escribe en YANG y se **compila** a un schema XML que dice exactamente qué campos existen, qué tipos tienen, cuáles son obligatorios y cuáles pueden repetirse.

```
Los módulos YANG de un fabricante:

  ietf-interfaces      → interfaces, susounteres, sus estados
  ietf-ip              → direcciones IPv4
  ietf-routing         → tabla de rutas estáticas
  ietf-system          → hostname,Clock, contactos
  cisco-ios-xe-*      → las extensiones del fabricante
```

La diferencia conceptual que hay que decir en el examen:

> SNMP **no define qué significa la información**: el agente publica una MIB y el gestor la interpreta. El **contenido semántico lo pone el fabricante** (o sea, **cada familia de equipos tiene su propio dialecto**).
>
> YANG **sí define la semántica**: el modelo es público, está escrito en un lenguaje estándar y describe **cómo se llama cada campo, de qué tipo es y cómo se relaciona con los otros**. **No dependés del fabricante para entender la configuración.**

Esa es **toda** la diferencia en una frase: **SNMP describe el dispositivo, YANG describe el modelo del mundo**.

La comparación que resume la clase:

	SNMP	NETCONF + YANG
Para qué	**monitoreo** (métricas)	**configuración** y telemetría
Modelo	MIB: un **árbol de variables**	YANG: un **lenguaje de modelado**
Transporte	UDP 161/162	**SSH** 830 (+ RESTCONF 443)
Modelo de datos	define **el dispositivo**	define **el modelo**, no el dispositivo
Semántica	**la pone el fabricante**	**estándar y pública**
Operaciones	genéricas (`GET` , `SET`)	RPCs con **significado** (`<lock>`)
Transaccional	no	**sí** (`<commit>`)
Seguridad	community strings (v2c) / USM (v3)	**SSH**: autenticado y cifrado
Push	traps (poco confiable)	**telemetría** (gNMI, YANG-Push)
Push vs pull	predomina **pull**	**push** estructurado

Lo que dice la propia IETF, textual (y está muy bien que lo tengas)

El grupo de trabajo de NETCONF escribió en 2024:

> *"SNMP & MIBs nunca alcanzaron tracción significativa en la industria para configuración… esperamos una disminución significativa de despliegues de SNMP en los próximos 10 años."*

Y lo que pasó en la realidad:

```
SNMP   →  SIGUE SIENDO EL REY DEL POLLING DE MÉTRICAS
          (Nagios, Zabbix, LibreNMS, Prometheus+SNMP Exporter, todo
           el mundo monitorea con SNMP)

NETCONF/YANG →  EL REY DE LA CONFIGURACIÓN Y DE LA TELEMETRÍA
          (está en Cisco IOS XE/NX-OS, Juniper Junos, Arista EOS,
           Nokia SR OS, Huawei VRP)
```

El reencuadre honesto que hay que poder escribir en el examen:

> *SNMP no está muerto: está desplazado. Se inventó para gestionar, y cumplió; pero el mundo descubrió que la parte de* preguntar métricas *era la fácil y la parte de* configurar *era la que siempre estuvo caída, porque el modelo no era suficientemente rico. NETCONF+YANG no reemplazó a SNMP: le sacó la mitad difícil y le dejó la mitad que funciona bien.*

Los otros que hay que mencionar (para sumar puntos)

Protocolo	Para qué	RFC
gNMI (*gRPC Network Management Interface*)	telemetría **push** estructurada, de Google, en los vendors grandes	no-IETF (gRPC sobre HTTP/2)
BMP (*BGP Monitoring Protocol*)	monitorear **BGP desde afuera** de un router	`RFC 7854`
IPFIX	exportar **flujos** (Flow Records)	`RFC 7011`
NTSF (*Network Telemetry Framework*)	**SPDX** y modelo común de telemetría	`RFC 9232`
syslog	eventos; sigue vivo y se usa **mucho**	`RFC 5424` (transporte TLS: `RFC 5425`)
NETCONF SSH: transporte, `RFC 6242` · **RESTCONF**: HTTP, `RFC 8040`		

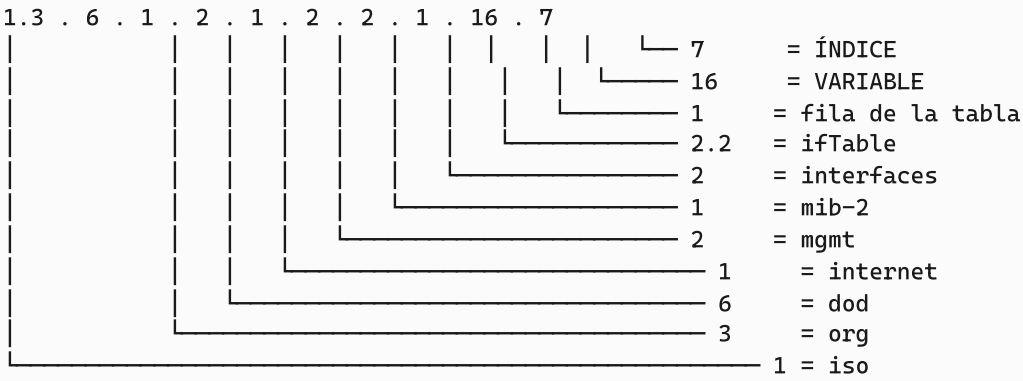
> **Un detalle que suma: NETCONF solo usa SSH**, pero **RESTCONF** (el mismo modelo YANG, sobre HTTP/HTTPS, puerto 443) es el que se usa para hablar con controladores que pasan por proxies y firewalls. Es el mismo modelo de datos, distinto transporte: **YANG es el modelo, NETCONF y RESTCONF son dosolateral**.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Leer un OID y encontrar la variable

Enunciado. Un gestor consulta `1.3.6.1.2.1.2.2.1.16.7`. Decí: ¿qué variable es, en qué tabla está, y qué interfaz representa?

Resolución paso a paso.



Parte	Significado
`1.3.6.1.2.1`	**MIB-II** (toda la gestión de Internet)
`.2`	el grupo `interfaces`
`.2.1`	la `ifTable`
`.16`	la **variable**: `ifOutOctets` — octetos **salientes** en la interfaz `ifIndex = 7`
`.7`	`ifIndex = 7`: la **séptima** interfaz del dispositivo

Interpretación: el gestor está leyendo **el total de bytes que la interfaz 7 mandó desde que arrancó**. Y como `ifOutOctets` es un **contador acumulativo de 32 bits que se desborda**, el valor **no esbytes por segundo**: es un **contador monótono creciente**.

Y acá está el error clásico que vale para el examen:

> ⚠️ **Un contador de 32 bits se desborda a los ~4 Gbytes.** Si el enlace lleva 1 Gbps, se desborda cada **32 segundos**. Por eso **nunca se consulta el valor y se lo interpola**: lo correcto es tomar **dos muestras separadas por un intervalo** y hacer la resta:
>
> $\text{tasa (bytes/s)} = (\text{ifOutOctets}_2 - \text{ifOutOctets}_1) / (t_2 - t_1)$
>
> Y si la diferencia da **negativa**, **no es un error de medición**: es el desbordamiento. Hay que sumarle $2^{32} = 4\ 294\ 967\ 296$ al resultado.

2.2 Ejercicio 2 —SNMPv3: el mensaje de una `GET` con USM

Enunciado. Escribí los campos de seguridad de un `GET` de SNMPv3 cuando el **receptor** es la autoridad (es decir, el gestor le pregunta al agente).

Resolución. Con USM, el mensaje lleva estos campos **en el nivel de seguridad de SNMP**:

```
┌──────────────────────────────────────────────────────┐
│  msgVersion                  = 3                     │
│  msgGlobalData                                       │
│     msgID                     = n (número de transacción)│
│     msgMaxSize                = 65507                │
│     msgFlags                                         │
│        authFlag  = 1     → HAY autenticación         │
│        privFlag  = 1     → HAY cifrado               │
│        reportableFlag = 1                            │
│     msgSecurityModel         = 3  (USM)              │
│  msgSecurityParameters                               │
│     msgAuthoritativeEngineID = ID del receptor  ← ★  │
│     msgUserName               = "admin"              │
│     msgAuthenticationParameters  = HMAC-SHA-256-128(...)  │
│     msgPrivacyParameters         = AES-192-CFB(...)  │
│     msgSecurityParameters        = las longitudes de los   │
│                                    anteriores, codificadas  │
└──────────────────────────────────────────────────────┘
+ PDU: GetRequest con el OID pedido
```

Lo que hay que explicar de cada campo:

1. `msgAuthoritativeEngineID` **= ID del receptor.** Porque la autoridad en SNMPv3 es siempre **quien responde**: el gestor le pregunta al agente y el agente es la autoridad. Y el ID del motor lo **sincroniza el descubrimiento**: el primer mensaje es una **solicitud de descubrimiento**, sin parámetros de seguridad, y la respuesta trae el `engineID` y los **algorithms** disponibles.
2. `msgFlags`**:** `authFlag` **y** `privFlag`**.** Los dos pueden ir por separado. Con solo `authFlag` tenés **autenticación e integridad pero no confidencialidad**: el contenido se ve en claro pero nadie lo puede modificar sin que se detecte. Es una opción real para datos que no son secretos pero sí deben ser íntegros.
3. **El HMAC va sobre el mensaje entero**, con los parámetros de seguridad puestos a cero y con una **clave privada del receptor** derivada de la contraseña. Por eso un HMAC calculado con una **clave compartida** no es igual al cálculo con la contraseña directa: se deriva primero (`password-to-key`), por lo mismo que en WPA.

2.3 Ejercicio 3 — Calcular el coste del polling vs. los traps

Datos. Red con **300 switches, 80 routers y 2 AP** = **382 dispositivos**. Se hace polling con **25 OIDs** por dispositivo, cada **60 segundos**.

Resolución — el polling:

```
Consultas por ciclo  = 382 dispositivos × 25 OIDs = 9 550 GETs
Por segundo          = 9 550 / 60 s = 159 GET/s
Por hora             = 159 × 3 600 = 573 000 GETs
Por día              = 573 000 × 24 = 13 752 000 GETs

Coste por paquete: un GET con un OID son ~100 bytes de payload,
más ~40 de UDP/IP/Ethernet = ~140 bytes.

Volumen diario = 13 752 000 × 140 B = 1,925 GB/día
                 ≈ 1,9 GB solo de SONDEO, todos los días.
```

Y con `GETBULK`, que es lo que se usa de verdad: `GETBULK` pide **25 OIDs en un paquete**. Entonces:

```
Paquetes por ciclo = 382
Por día            = 382 × 1 440 ciclos/día = 550 080 paquetes/día
Volumen            = 550 080 × ~1 800 B (25 OIDs) ≈ 990 MB/día

→ 25 veces MENOS tráfico, misma información. ★
```

Interpretación: ese es el motivo de ser de `GETBULK`, y por eso **existe en v2c y no en v1**: v1 era inmanejable y no se podía monitorear nada grande.

Los traps, en comparación:

```
Coste de un trap: ~1 paquete cuando HAY evento, nada cuando no hay.
Límite: 2 traps/s por agente.

El peor caso teórico: 382 × 2 = 764 traps/s. Pero eso es
un dispositivo que está fallando masivamente, que es exactamente
cuando QUERÉS que el sistema grite.
```

La conclusión del ejercicio: el polling es **predecible** pero **caro y ciego**; el trap es **barato y reactivo** pero **se pierde si el gestor está caído**.

2.4 Ejercicio 4 — Convertir un comando de CLI en NETCONF

Enunciado. El SNMP `SET` es inútil para configurar. Comparar: `snmp-server community public RO` (SNMP) vs. la misma intención en NETCONF.

Resolución — el mismo objetivo por dos caminos:

```
SNMP v2c                                NETCONF + YANG
────────                                ──────────────
snmp-server community public RO        system {      ← módulo YANG "ietf-system"
  (se ejecuta en la CLI del agente,        host-name "r1.miuy.edu.uy";
   NO ES UN GET/SET: es una instrucción   location    "Piso 3, Sala 302";
   de configuración del agente, escrita    contact     "noc@miuy.edu.uy";
   a mano por el admin)                  }
```

Resolución con el XML del `<edit-config>`:

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <edit-config>
    <target>
      <running/>
    </target>
    <default-operation>merge</default-operation>
    <config>
      <system xmlns="urn:ietf:params:xml:ns:yang:ietf-system">
        <host-name>r1.miuy.edu.uy</host-name>
        <location>Piso 3, Sala 302</location>
        <contact>noc@miuy.edu.uy</contact>
      </system>
    </config>
  </edit-config>
</rpc>

<rpc-reply message-id="101">
  <ok/>
</rpc-reply>
```

Y con datastores, que es la diferencia real:

```
<!-- 1. editar el BORRADOR, sin tocar nada en ejecución -->
<edit-config>
  <target><candidate/></target>
  <config> ... las 20 líneas nuevas ... </config>
</edit-config>

<!-- 2. VALIDAR sin aplicar: si algo está mal, NO cambió nada -->
<validate><source><candidate/></source></validate>

<!-- 3. si el validate da OK, aplicar de golpe -->
<commit/>
```

Lo que gained con esto: si la línea 15 de las 20 es inválida, con `commit` **no se aplica ninguna**; con 20 `SET` de SNMP **se aplicaron 14** y el equipo quedó con una configuración **mixta, a medias y no deseada**.

2.5 Ejercicio 5 — Escribir una MIB mínima

Enunciado. Escribí el fragmento ASN.1/SMIv2 de una MIB que defina una variable `turnosPendientes` que cuente los pedidos de turno en espera.

Resolución:

```
-- ================================================
-- MIB-MIYUY-MIB
-- ================================================
TURNOS-MIB DEFINITIONS ::= BEGIN

IMPORTS
    MODULE-IDENTITY, OBJECT-TYPE, Integer32, Counter32, TimeTicks
        FROM SNMPv2-SMI
    enterprises, MODULE-COMPLIANCE, OBJECT-GROUP
        FROM SNMPv2-SMI;

miyuyTurnos MODULE-IDENTITY
    LAST-UPDATED "202610010000Z"       -- formato AAAAMMDDHHMMSSZ
    ORGANIZATION "Facultad de Ingenieria, UdelaR"
    CONTACT-INFO "noc@miuy.edu.uy"
    DESCRIPTION  "Gestion de la cola de turnos de la cafeteria."
    ::= { enterprises 99999 }        -- 99999 = numero MI (declarar el propio)

miyuyTurnosMIBObjects  OBJECT IDENTIFIER ::= { miyuyTurnos 1 }

-- El objeto: un contador
turnosPendientes OBJECT-TYPE
    SYNTAX      Counter32          -- se incrementa, NUNCA baja
    UNITS       "turnos"           -- la unidad (conteo, bytes, segundos)
    MAX-ACCESS  read-only          -- no escribible desde afuera
    STATUS      current
    DESCRIPTION "Cantidad de personas esperando turno en este momento.
                 Es monotono creciente entre reinicios del agente:
                 cuando se llega a 2^32 el agente lo vuelve a cero."
    ::= { miyuyTurnosMIBObjects 1 }

-- El objeto: un estado
estadoTurnos OBJECT-TYPE
    SYNTAX      INTEGER {
                   normal(1),
                  IEF(2),    -- (con un typo, a proposito, para mostrarlo)
                   saturado(3),
                   cerrado(4)
               }
    MAX-ACCESS  read-write          -- este SÍ se puede cambiar
    STATUS      current
    DESCRIPTION "Estado operativo del sistema de turnos."
    ::= { miyuyTurnosMIBObjects 2 }

miyuyTurnosCompliance MODULE-COMPLIANCE
    STATUS current
    MODULE  -- este módulo
    ::= { miyuyTurnosMIBObjects 10 }

END
```

Lo que hay que mirar en ese fragmento, porque cada detalle es una pregunta de examen:

Detalle	Por qué importa
`LAST-UPDATED "202610010000Z"`	formato **obligatorio** `AAAAMMDDHHMMMMZ`; si no cambia, el OID **nunca** se puede reutilizar
`Counter32` vs. `Integer32`	`Counter32` **sube y se desborda**; `Integer32` **sube y baja**. Un contador de bytes **jamás** debe ser `Integer32`
`MAX-ACCESS read-only` vs. `read-write`	`read-write` **es una superficie de ataque**: cualquiera que pueda escribir puede **cambiar la configuración** del equipo por SNMP
`STATUS current`	`current` = vigente · `deprecated` = obsoleto pero se usa · `obsolete` = **no usar**
`::= { enterprises 99999 }`	hay que **registrar el número de empresa** ante IANA
El módulo `SNMPv2-SMI`	**SMIv2**; SMIv1 (RFC 1155) no tiene `MODULE-IDENTITY` ni `Counter32`

2.6 Ejercicio 6 — SNMPv3 con VACM: dimensionar una vista

Enunciado. En una red de la facultad hay tres perfiles de usuario: `sensor`, `operador` y `admin`. Definir las vistas.

Resolución — el razonamiento por perfil:

```
PERFIL     ¿QUÉ NECESITA VER?                  ¿QUÉ DEBE PODER HACER?
─────────  ──────────────────────────────────  ──────────────────────

sensor     temperature, estado de las          sólo GET / GETNEXT.
           interfaces, uso de puertos.         ← NUNCA SET.

operador   todo lo de sensor + tablas de       GET, GETNEXT, GETBULK
           ARP e interfaces; sysUpTime;        y SET **sólo sobre
           ipAddressTable de SU subred.        la subred propia.

admin      todo (1.3.6.1.2.1.*)                GET, SET, y también
                                                 `<lock>`/`<unlock>` para
                                                 no pisar a otro.
```

```
<!-- Vista del sensor: SOLO lectura, y SOLO las subárboles termometricos -->
<snmp-view view sensor-subtree included subtree 1.3.6.1.2.1.14.*/>
<snmp-view view if-subtree    included subtree 1.3.6.1.2.1.2.2.*/>

<access-group group sensor-ro version v3 security-level authPriv
              context "" match exact read-view sensor-subtree
              read-view if-subtree
              write-view ""                          ← vacío = no puede escribir
              notify-view "" />

<!-- Vista del operador: incluye la subnet de cada uno -->
<snmp-view view routing-subtree included subtree 1.3.6.1.2.1.4.*/>
<snmp-view view own-net       included subtree 1.3.6.1.2.1.4.20.1.<SUBNET>.0>

<!-- Vista del admin: todo -->
<snmp-view view all included subtree 1.3.6.1.2.1/>
```

La justificación que hay que dar (es lo que suma puntos):

1. **El `write-view ""` (vacío) del sensor es la decisión importante.** Sin eso, un sensor térmico compromise puede **cambiar la configuración del switch** y dejar la red abajo. Mínimo privilegio.
2. **Quitar `ipRouteTable` al sensor no es cortesía, es seguridad.** La tabla de ruteo dice **dónde está cada red**: es exactamente lo que un atacante necesita para pivotar. Lo mismo con `ipNetToMediaTable` (la tabla ARP).
3. **`authPriv` (autenticación + cifrado) y no solo `auth`.** Los datos de gestión son datos **sobre tu infraestructura**: si viajan en claro, cualquier sniffer los lee y además puede **inyectar respuestas falsas** (*spoofing*) y hacer que el gestor se cuelgue (el ataque más clásico: matar el SNMP y que nadie crea que la red está mal).

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2010-03, Pregunta 4 (8 puntos) — agente, gestor y MIB

> **Fuente:** Examen `sirc1003`, 24 de febrero de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1003.pdf **Texto del examen:** *En el contexto de la gestión de las redes de computadoras, a) Explique los conceptos "agente" y "gestor". b) Describa brevemente el modelo de información MIB (Management Information Base).*

Resolución

a) Agente y gestor (4 puntos)

Son dos papeles del mismo modelo, y definirlos es la mitad de la pregunta.

El agente es la entidad de software **residente dentro del sistema que se gestiona** (el switch, el router, la AP, el servidor). Su trabajo es doble:

1. **Colectar** información de gestión del dispositivo donde vive. Y como vive adentro, tiene **recursos mínimos**: los de ese equipo. Un switch de acceso tiene CPU lenta y memoria contada, así que el agente está escrito para ser **pequeño**.
2. **Generar notificaciones** (*traps*, alarmas) cuando ocurre algo.

Y lo que **no** hace, que es la parte que hay que decir explícitamente porque es lo que distingue al agente del gestor: **no toma decisiones**. El agente puede coleccionar, pero **no decide** en función de lo que colecta. *(Excepción clásica: RMON, donde el agente guarda estadísticas históricas en el propio dispositivo para no traerlas.)*

El gestor (o **NMS**, *Network Management Station*) es la entidad que **recoge la información de todos los agentes** y es **la interfaz de usuario** del administrador. Sus rasgos:

- **Es la inteligencia de la gestión.** Correlaciona, decide y dispara acciones.
- **Hay un gestor para muchos agentes.** La relación es de **1 a N**: un gestor, 2 000 agentes.
- **Guarda histórico**, que es lo que le permite decir "el puerto 7 estaba al 95 % entre las 3 y las 4 AM del martes".

Y el rasgo que la cátedra menciona porque es lo que hace la gestión **real**: **interoperabilidad entre gestores**, o sea **gestión integrada y distribuida**. Cuando la red es tan grande que un solo gestor no alcanza, varios gestores se hablan entre sí (con SNMP y con peerings como BGP): uno queda a cargo del site A y otro del site B, y se contunden información. Hoy esto se llama **FedRAMP**... digo, **federación** y es la base de la gestión multi-tenant.

> **El paralelo con los routing protocols, que es lo que más suma:** es **exactamente** la relación entre un **protocolo de enrutamiento** y un **sistema autónomo**. El agente es como un router que **anuncia** su estado, y el gestor es como un router que **recibe anuncios y calcula** una tabla global. Y por eso

> aparecen los mismos problemas: **agente contra agente** (hay que autenticar, o el *spoofing* es trivial) y **gestor contra gestor** (hay que decidir quién manda).

b) El modelo MIB (4 puntos)

El problema que la MIB resuelve es la interoperabilidad, y esa es la primera frase de la respuesta:

> *El agente y el gestor pueden ser de diferentes proveedores y de diferentes versiones. Sin un modelo de información común y estandarizado, un gestor de un fabricante no podría entender lo que publica el agente de otro. Por eso hace falta un modelo de información de gestión estándar: ese modelo es la MIB.*

Cómo está construida:

1. **Todo recurso de red gestionable se representa como un objeto.** Y un objeto, en este contexto, es simplemente **una variable que contiene información del recurso**. El tipo de la variable depende del recurso (un contador de bytes, un estado, un nombre, una dirección).
2. **El conjunto de todas las variables conocidas por un agente es la MIB de ese agente.** Ojo con la frase: la MIB **es del agente**, no de la red. Cada agente implementa un subconjunto de la MIB-II según de qué unidad tiene.
3. **El gestor accede a los objetos presentes en la MIB de cada agente.** Y — la parte clave del punto (b) de la solución oficial — **lo que se puede hacer con cada agente depende de la MIB implementada por ese agente**: si el firmware no trae la MIB de BGP, el gestor **no tiene forma de saber** los indicadores de BGP, porque **no hay con qué preguntar**.
4. **Cada variable tiene un OID** (*Object Identifier*), que es una **dirección jerárquica** en un árbol. Los OIDs se asignan por la autoridad (**IANA**), que delega a los proveedores, que a su vez delegan a los fabricantes. De ahí la estructura: `1.3.6.1.2.1.*` es **MIB-II** (todo IPv4) y `1.3.6.1.4.1.*` es **empresa** (cada fabricante con su número).
5. **La MIB se escribe en ASN.1**, y el SNMP usa **SMIv2** (ya en desuso) o **SMIv2** (`SNMPv2-SMI`), que agrega `MODULE-IDENTITY`, `Counter32`, `Gauge32`, `TimeTicks` y los `MAX-ACCESS` con los valores correctos.

El detalle fino que suma puntos: la MIB **es un archivo de texto** (`.mib`, `.my`). **No se instala en el equipo.** El agente ya tiene compilados los valores de esas variables; la MIB es **la documentación que le sirve al gestor para saber qué números significan qué**. Por eso el mismo equipo con el mismo firmware puede monitorearse con cualquier gestor, siempre y cuando el gestor tenga la MIB cargada.

Lo que NO escribió la solución oficial y suma

La solución oficial es correcta pero se queda en "qué es". Para el examen completo agregá:

- **Que la MIB es un modelo de *variables, no de acciones***: describe el estado del dispositivo, no lo que puede hacer. Por eso configurar por SNMP es deuda: el `SET` existe, pero **no hay semántica de configuración**, no hay transacción y no hay validación. Ese es el motivo por el que existe NETCONF.

- **Que hay MIB-II (`1.3.6.1.2.1`) y las MIB propias de cada fabricante** (`1.3.6.1.4.1.<empresa>.*`), y que las de fabricante **no son portables**.
- **Que el OID se negocia con `GETNEXT`** y no se pide por nombre (ver §1.5).

3.2 Examen 2008-07, Pregunta 4 (4 puntos) — planos de datos y de control

> **Texto del examen:** *Compare el plano de datos y el plano de control, e indique si la comunicación de gestión se realiza en banda o fuera de banda.*

Resolución. Es la pregunta que más se olvida y es la base conceptual de toda esta clase.

```
┌──────────────────────────────────────────────────────────┐
│  PLANO DE DATOS                                          │
│  Los paquetes del usuario: IP, TCP, UDP, HTTP...         │
│  Es lo que el equipo hace PARA CARRear el tráfico.       │
│  Alta tasa, latencia baja, contenido variable.           │
├──────────────────────────────────────────────────────────┤
│  PLANO DE CONTROL                                        │
│  Los mensajes de gestión y de señalización:              │
│  ARP, ICMP, OSPF, BGP, SNMP, STP, DHCP, CDP, LLDP.       │
│  Es lo que el equipo hace PARA CONFIGURARSE y saber del  │
│  estado de la red.                                       │
│  Baja tasa, a veces con requisitos de tiempo.            │
└──────────────────────────────────────────────────────────┘
```

¿En banda o fuera de banda?

```
EN BANDA (in-band)                  FUERA DE BANDA (out-of-band)
──────────────────                  ────────────────────────────
El tráfico de gestión viaja         El tráfico de gestión viaja
por la MISMA red que el tráfico     por una RED SEPARADA, dedicada
de los usuarios.                    (o por un enlace de consola).

Se administra por SSH/telnet        Se administra por consola
desde cualquier lugar.              física (RJ-45), que funciona
                                    AUNQUE la red esté caída.

★ Si la red está caída,              ★ Si la red está caída,
NO se puede administrar.            SE PUEDE administrar igual.

Ejemplo: NOC compartido, acceso      Ejemplo: consola del rack,
por VPN a cualquier equipo.         El cable de consola del rack.
```

La respuesta completa: el tráfico de gestión **es del plano de control**, y **en banda o fuera de banda depende de la criticidad del activo**. En una red universitaria, el router del NOC y los switches de backbone **deberían** tener **fuera de banda** (consola + un puerto de gestión en una VLAN dedicada), porque si se cae el enlace principal y no tenés fuera de banda, **perdiste el control de la red justo cuando más lo necesitás**.

Por qué importa de verdad, y no es teórico: es el motivo por el que existe **VLAN de gestión**, **VLAN de voz**, y por qué los equipos de red tienen un **puerto de consola RJ-45** (que **no es un puerto de datos**: no lo conectes a un switch, porque no es un enlace Ethernet, es una consola asíncrona — y a veces es USB, no RJ-45, y el cable es **rollover**).

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. La gestión de redes existe porque `ping` **no escala**, no da **métricas** y no guarda **histórico**. Las cinco funciones: *failure, configuration, accounting, performance, security*.
2. **Agente** = programa chico que **dentro** del equipo **colecciona** y **responde**, pero **no decide**. **Gestor** = central, con recursos, **decide todo**. La asimetría es el corazón de la pregunta.
3. **MIB** = la base de datos de variables del agente, con **OIDs jerárquicos** (`1.3.6.1.2.1.*` = MIB-II; `1.3.6.1.4.1.*` = fabricante), escrita en **ASN.1/SMIv2**.
4. La MIB es **un archivo de texto que se instala en el gestor, no en el equipo**: el equipo ya tiene los valores, el gestor necesita saber qué significan.
5. **Polling** (pull: `GET`, `GETNEXT`, `GETBULK`) = caro, predecible, pero **muerto si el gestor se cae**. **Traps** (push) = barato, instantáneo, pero **se pierden si el gestor se cae**. **Se usan los dos.**
6. `GETNEXT` es **un puntero que recorre el árbol**; `GETBULK` es lo mismo, **N de ellos en un paquete** (reduce 25× el tráfico). Existe **por eso**, y es de v2c.
7. Un `Counter32` **se desborda**: la tasa es $(v_2 - v_1) / (t_2 - t_1)$, y si la resta es **negativa** hay que sumarle 2^{32}.
8. **SNMPv3 = USM (autenticación + cifrado, quién sos) + VACM (qué podés ver, mínimo privilegio por vistas)**.
9. **NETCONF = SSH + XML + RPCs con semántica + transacción (`candidate` → `commit`). YANG** define el **modelo**, no el dispositivo: por eso se entiende sin importar el fabricante.
10. **SNMP no murió: se desplazó.** Sigue siendo el rey del **polling de métricas**; NETCONF/YANG es el rey de la **configuración** y de la **telemetría push** (gNMI, YANG-Push, RESTCONF).

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué un agente SNMP no puede tomar decisiones, y por qué el gestor sí?

<details>
<summary>Respuesta</summary>
</details>

2. Explicá por qué `GETNEXT` existe y qué problema resuelve.

<details>
<summary>Respuesta</summary>
</details>

3. ¿Por qué un `Counter32` puede dar una resta negativa y qué hay que hacer?

<details>
<summary>Respuesta</summary>
</details>

4. ¿Por qué `write-view ""` para el sensor térmico? Explicá el razonamiento de seguridad.

▶ Respuesta

5. Diferenciá SNMP y NETCONF en una frase cada uno, y decí cuál no reemplazó al otro y por qué.

▶ Respuesta

6. Un switch tiene un puerto en estado `down` y no hay traps. ¿Qué se deduce?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1003` — Pregunta 4 completa (8 pts: agente, gestor y modelo MIB)
- **Examen** `sirc0807` — Pregunta 4 (4 pts: planos de datos/control, en banda vs. fuera de banda)
- **Examen** `sirc0712` — Pregunta 5 (MPLS: router de ingreso y de egreso, otra forma de separar planos)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 5 §5.5 (gestión de redes)
- `RFC 1157` — SNMPv1 · `RFC 3411` — SNMPv3 arquitectura · `RFC 3414` — USM · `RFC 3415` — VACM · `RFC 3416` — operaciones · `RFC 7860` — HMAC-SHA-2 en USM
- `RFC 6241` — NETCONF · `RFC 6242` — NETCONF sobre SSH · `RFC 7950` — YANG 1.1 · `RFC 8040` — RESTCONF · `RFC 8342` — NMDA · `RFC 8639` / `8640` / `8641` — YANG-Push · `RFC 7854` — BMP · `RFC 7011` — IPFIX · `RFC 9232` — Network Telemetry Framework
- `ACTUALIZACIONES.md` **§11** — el reencuadre SNMP vs. NETCONF/YANG
- **Clase siguiente:** 47 — Seguridad: servicios y criptografía